601.433/633 Intro Algorithms
Recitation 5

Fall 2026

Dynamic Programming

Suppose you are standing at the bottom of a staircase consisting of n steps, numbered $1, 2, \ldots, n$. Your starting position is step 0, and your goal is to reach step n.

At each move, you may climb either:

(a) Exactly one step, moving from step i to step $i + 1$, or

(b) Exactly two steps, moving from step i to step $i + 2$.

A valid way of climbing the staircase is a sequence of one-step and two-step moves that starts at step 0 and ends at step n. Two ways are considered distinct if their sequences of moves differ. In particular, the order of the moves matters. For example, if $n = 3$, there are exactly three distinct ways to reach the top: $(1, 1, 1)$, $(1, 2)$, and $(2, 1)$, where each number represents the number of steps climbed in that move.

The problem: Given a positive integer n, determine the total number of distinct ways to climb exactly n steps.

Design a dynamic programming algorithm to solve this problem. Prove its correctness and analyze its running time.

Solution:

Let S_i be the set of all sequences of moves in $\{1, 2\}$ whose lengths sum to i, and let $OPT(i) = |S_i|$. We compute a table $W[1], \ldots, W[n]$, intended to satisfy $W[i] = OPT(i)$. We first prove an optimal substructure theorem that will help us design our dynamic programming solution:

Theorem 1 (Optimal substructure theorem)**.** *For every $i \geq 3$,*

$$OPT(i) = OPT(i-1) + OPT(i-2).$$

Proof. We prove the equality by showing both $OPT(i) \leq OPT(i-1) + OPT(i-2)$ and $OPT(i) \geq OPT(i-1) + OPT(i-2)$.

Partition S_i into A_i, the sequences ending in a one-step move, and B_i, the sequences ending in a two-step move. Every legal sequence ends in exactly one of these moves, so the two sets are disjoint and

$$OPT(i) = |A_i| + |B_i|.$$

$\leq$: Removing the last move of every sequence in A_i maps A_i into S_{i-1}. This map is injective: if two sequences in A_i become equal after their final one-step moves are removed, the original sequences were equal. Therefore $|A_i| \leq |S_{i-1}| = OPT(i-1)$. Analogously, removing the last two-step move of every sequence in B_i injectively maps B_i into S_{i-2}, so $|B_i| \leq |S_{i-2}| = OPT(i-2)$. Hence

$$OPT(i) \leq OPT(i-1) + OPT(i-2).$$

$\geq$: Conversely, append a one-step move to each sequence in S_{i-1}. The resulting sequences are valid members of A_i; distinct prefixes give distinct sequences. Thus $|A_i| \geq |S_{i-1}| = OPT(i-1)$. Appending a two-step move to each sequence in S_{i-2} similarly gives $|B_i| \geq |S_{i-2}| = OPT(i-2)$. Therefore,

$$OPT(i) \geq OPT(i-1) + OPT(i-2).$$

Combining the upper and lower bounds proves the claimed equality. □

Algorithm. Set $W[1] = 1$ and, if $n \geq 2$, set $W[2] = 2$. If $n \geq 3$, then for $i = 3, \ldots, n$, compute $W[i] = W[i-1] + W[i-2]$. Return $W[n]$. This is a bottom-up dynamic programming algorithm:

```
ClimbStairs(n) {
  W[1] = 1;
  if n ≥ 2 then W[2] = 2;
  if n ≥ 3 then {
    for (i = 3 to n) {
      W[i] = W[i − 1] + W[i − 2];
    }
  }
  return W[n];
}
```

Theorem 2 (Algorithm Correctness)**.** *For every $1 \leq i \leq n$, the algorithm computes $W[i] = OPT(i)$; thus it returns the number of valid sequences reaching step n.*

Proof. We will prove this by strong induction on i. For $i = 1$, the only sequence is (1), so $W[1] = 1 = OPT(1)$. For $i = 2$, the distinct sequences are $(1, 1)$ and (2), so $W[2] = 2 = OPT(2)$. For the inductive step, assume for some $i \geq 3$ that $W[j] = OPT(j)$ for all $j < i$. By the algorithm and the induction hypothesis,

$$W[i] = W[i-1] + W[i-2] = OPT(i-1) + OPT(i-2).$$

By the optimal substructure theorem (Theorem 1), the right-hand side equals $OPT(i)$. Hence $W[i] = OPT(i)$, completing the induction. □

Running time calculation. We analyze the bottom-up algorithm above. Assigning $W[1]$ and $W[2]$ takes two constant-time assignments, so this costs $O(1)$. The loop executes at most $n-2$ times, and each iteration performs one addition and one table assignment, both of which take $O(1)$ time. Therefore, the running time is $O(n)$, and the DP table occupies $O(n)$ space.